

FLIPPED COURSE EVALUATION PROJECT REPORT

System Title: Personal Expense & Budget Planner CLI
Author: Yogendra | **Visibility:** Public Repository Ready

CHAPTER 1: COVER PAGE

Course Focus: Flipped Course Evaluation Project Portfolio Elements
Development Scope: Production-Grade Modular Personal Finance Accounting Tools
Technical Language: Native Programmatic Python (100% Dependency-Free Blueprint Pattern)
Target Repository Context: <https://github.com/yogendra26bme10008-byte/personal-budget-tracker>
Execution Environment Boundary: Terminal Command-Line Processing Window Interface Only

CHAPTER 2: INTRODUCTION

This computer software utility engine is carefully designed to provide real-time user-interactive personal financial tracking matrices directly inside low-overhead operating terminal shells. Unlike commercial desktop suites that demand permanent internet connectivity hooks or complex heavy graphical layers, this utility architecture prioritizes absolute logical isolation, optimal execution processing speeds, and fast file persistence routines to evaluate active net economic user profiles easily.

CHAPTER 3: PROBLEM STATEMENT

Modern students and developers frequently struggle to maintain stringently balanced expenditure constraints due to the configuration overhead of heavy graphical budgeting applications, alongside data security fears linked to passing private records to remote database servers. This system neutralizes these workflow challenges by presenting an instantly responsive local terminal navigation map that inputs, filters, and logs rows using high-speed, local file system string table arrays.

CHAPTER 4: FUNCTIONAL REQUIREMENTS

The system execution pipeline satisfies academic grading targets by implementing three distinct operational service cells:

- Module 1 (Budget Configuration Base):** Validates string-to-float conversions to securely lock an anchor spending ceiling threshold.
- Module 2 (Iterative Transaction Parsing):** Processes multi-variable inputs for pricing values, names, and custom text categories dynamically.
- Module 3 (Analytical Performance Summary):** Aggregates historical record blocks instantaneously to render visual balance statuses, savings numbers, or overage ceiling warning indicators.

CHAPTER 5: NON-FUNCTIONAL REQUIREMENTS

The program architecture adheres to exactly four required architectural constraints:

1. **Usability:** Plain numerical menu index parsing entries (Options 1 to 4) provide unambiguous navigational flow maps.
2. **Reliability:** Incorporates tight structural exception-catching (try-except) boundaries to eliminate runtime code exceptions.
3. **Maintainability:** Decouples functional business features across five distinct software component modules, minimizing dependency locks.
4. **Portability / Efficiency:** Relies exclusively on core built-in variables with zero third-party installation flags required for cross-platform execution.

CHAPTER 6: SYSTEM ARCHITECTURE

The software uses an decoupled, n-tier Layered System Architecture layout format:

- *User Interface Interaction Hook:* main.py handles continuous menu interface loops.
- *Operational Logic Processing Engine:* core.py and dashboard.py calculate numerical variances.
- *Data Storage & Format Layer:* storage.py and config.py map stream parsing schemas.
- *Hardware Layer Bound:* expenses.txt maps raw rows securely onto local memory segments.

CHAPTER 7: DESIGN DIAGRAMS

A. Process / Workflow Map Blueprint Sequence:

[App Boot] -> [Read lines expenses.txt] -> [Render Menu Choice 1-4]

■■■ Choice 1: Update Target Floor -> Save back to stream

■■■ Choice 2: Input Item Row -> Validate Bounds -> Append Array

■■■ Choice 3: Compile Outlay Aggregate -> Deduct from Limit -> Print Savings

B. Component Dependency Schema Blueprint:

[main.py (Runtime Execution Shell)] --> [core.py (Calculations Core)]

■■■> [dashboard.py (Report Builder)]

■■■> [storage.py (I/O Module)] --> [config.py]

CHAPTER 8: DESIGN DECISIONS & RATIONALE

- *Decision:* Employing customized plain-text parsing characters over heavy structured JSON file parsers.
- *Rationale:* Eliminating internal package loading requirements speeds up raw terminal execution frames, maintains transparent user validation checks, and strictly guarantees clean software isolation compliance properties.

CHAPTER 9: IMPLEMENTATION DETAILS

The system execution directory contains exactly 5 decoupled source blocks:

- **config.py**: Houses constant pointers (`DATA_FILE = 'expenses.txt').
- **storage.py**: Contains target low-level byte stream encoding structures.
- **core.py**: Hosts conditional logic checking limits values dynamically.
- **dashboard.py**: Organizes data records cleanly into output summary charts.
- **main.py**: Drives the global app workflow experience within an indefinite loop block.

CHAPTER 10: SCREENSHOTS / RESULTS

Example System Dashboard Report Generated by Terminal Command Selection 3:

```
=====
                MY PERSONAL BUDGET REPORT
=====
■ Total Budget      : ■1000000.00
■ Total Money Spent : ■122222.00
-----
■ TOTAL SAVINGS     : ■877778.00
-----
EXPENSE HISTORY:
1. [3] 454545 - ■122222.00
=====
```

CHAPTER 11: TESTING APPROACH

System validations involved critical parameter checks:

- *Malformed String Interception*: Entering textual data string maps into the option number inputs. The exception boundaries intercepted the anomaly safely and reverted back to menu choices smoothly.
- *Overage Bound Triggering*: Injecting pricing values that pass active budget points safely logs the item and alerts the terminal screen with an automated warning header.

CHAPTER 12: CHALLENGES FACED

- *Challenge*: Satisfying the multi-module 5-file requirement without access to complex dictionary databases.
- *Mitigation Path*: Built custom linear record-splitting string engines using native Python `.strip().split('|')` pipelines to maintain structured rows flawlessly.

CHAPTER 13: LEARNINGS & KEY TAKEAWAYS

Acquired sharp skills regarding procedural code file division patterns, error-catching boundary closures, local storage array tracking, and execution thread management maps in isolation. Gained deep understanding of production-grade program organization basics.

CHAPTER 14: FUTURE ENHANCEMENTS

- Integrate automatic system time-series stamps onto saving fields.
- Incorporate terminal visual data graphing models (e.g., lightweight string-based histograms).
- Introduce text-matching regex queries to filter expenses by exact keywords inside categories.

CHAPTER 15: REFERENCES

1. Python Software Foundation Documentation Guide (File I/O Streams Processing Architecture Manual).
2. Structural Decoupling Patterns in Modern Software Architecture Standards, University Technical Syllabus Specification Texts.

This is for informational purposes only. For medical advice or diagnosis, consult a professional. AI responses may include mistakes.